
```
def displayBoard(board):
    """Display the board and its tiles on the screen."""

    # Prepare a list to pass to the format() string method for the board
    # template. The list holds all of the board's tiles (and empty
    # spaces) going left to right, top to bottom:
    tileChars = []
```

Recall that the `BOARD_TEMPLATE` is a multiline string with several brace pairs. When we call the `format()` method on `BOARD_TEMPLATE`, these braces will be replaced by the arguments passed to `format()`.

The `tileChars` variable will contain a list of these arguments. We start by assigning it a blank list. The first value in `tileChars` will replace the first pair of braces in `BOARD_TEMPLATE`, the second value will replace the second pair, and so on. Essentially, we're creating a list of the values from the `board` dictionary:

```
for rowIndex in range(BOARD_HEIGHT):
    for columnIndex in range(BOARD_WIDTH):
        tileChars.append(board[(columnIndex, rowIndex)])

# Display the board:
print(BOARD_TEMPLATE.format(*tileChars))
```

These nested `for` loops iterate over every possible row and column on the board, appending them to the list in `tileChars`. Once these loops have finished, we pass the values in the `tileChars` list as individual arguments to the `format()` method using the star `*` prefix. “Using `*` to Create Variadic Functions” section on page 167 explained how to use this syntax to treat the values in a list as separate function arguments: the code `print(*['cat', 'dog', 'rat'])` is equivalent to `print('cat', 'dog', 'rat')`. We need the star because the `format()` method expects one argument for every brace pair, not a single list argument.

Next, we write the `getPlayerMove()` function:

```
def getPlayerMove(playerTile, board):
    """Let a player select a column on the board to drop a tile into.

    Returns a tuple of the (column, row) that the tile falls into."""
```
